Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

WiBalance – Intelligent Wi-Fi Traffic and Load Balancing System

Team Name: *Team #*	*Net Ninjas #DSCPP-III-2026-T289*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Chauhan, Aakriti – 2510380277* *aakriti1301chauhan@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Vashishth, Vaibhav – 2510350076* *Vaibhavvashishth378@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Raj, Khush – 2510380255* *khushrajsinghranawat207@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

We've all been there on campus or office Wi-Fi. Your phone shows full bars, you're connected, and yet pages crawl. A lot of the time the culprit isn't the signal at all. It's that dozens of people have piled onto one Access Point (AP) while the one down the corridor sits nearly empty.
That's the problem WiBalance is meant to solve, and we're writing it in C++.
WiBalance checks more than just the number of users connected to an AP. The type and amount of traffic also affect its load. For instance, a few users downloading large files can put more load on an AP than several users who are only sending messages. To check this, our program uses bandwidth, throughput, latency, packet loss, signal strength and channel usage. It uses these values to calculate the load of each AP. When an AP gets too busy, the program looks for another AP with less load and shifts users to it.

Updated Project Approach and Architecture

We have kept the main plan from Phase 1, but we are starting with simulated APs and users instead of connecting real Wi-Fi hardware. Real AP integration and ns-3 are optional for this phase. We focused on getting the C++ core working first, so the Python dashboard is still planned for the next phase.
Here's how it runs. The program sets up simulated APs and users as C++ classes and keeps track of their metrics. A weighted formula combines those numbers into a single load score for every AP. Once a score crosses the threshold, we flag that AP as congested and move a few of its users to a better one. We'll judge how well this works by looking at latency, packet loss and AP utilization.

Tasks Completed

Task Completed	Team Member
AP metrics: load %, channel utilization, latency and packet loss	Aakriti Chauhan
C data-structure module (`dsa.c`, `dsa.h`): queue, hash table, search and insertion sort	Aakriti Chauhan
Weighted AP selection score (load, signal, latency, client count, priority, mobility)	Vaibhav Vashishth
Congestion detection (HEALTHY / MODERATE / CONGESTED) and tick-based redistribution	Vaibhav Vashishth
C++ classes `User`, `AccessPoint` and `WiBalance` with simulated traffic	Khush Raj Singh
Menu-driven interface with input validation	Khush Raj Singh

Challenges/Roadblocks

- **Choosing the weights:** Deciding how much each metric should count was harder than we expected. We ended up trying several combinations and comparing what each one did to the load scores.
- **Making fake traffic look real:** Since we don't have real APs, the simulated traffic had to behave like actual usage, with sudden rushes and long quiet stretches.
- **Users bouncing around:** In early runs, the same users kept getting moved from one AP to another, again and again. We're now adding a cooldown between moves to stop that.
- **Working as a team:** Sharing the code between us was fine, but fitting it around exams and other subjects was tricky.
- **Learning ns-3:** It ate up more time than we planned for, so we made it optional instead of forcing it into this phase.

Tasks Pending

Task Pending	Team Member
Performance metrics: average latency, packet loss and AP utilization	Aakriti Chauhan
More balancing algorithms (e.g. Round Robin) and a cooldown between user moves	Vaibhav Vashishth
QoS manager for priority traffic	Vaibhav Vashishth
CSV storage and event log files	Khush Raj Singh
Python dashboard, further testing and final documentation	All members

Project Outcome/Deliverables

At this point we have a working C++ program that simulates several APs and users. Each user has a traffic type, a demand and a location, and each AP reports its own load, channel use, latency and packet loss. From these numbers the program works out a weighted score, spots the congested APs and moves users over to less busy ones. Everything runs from a simple menu, and the queue, hash table, search and sort behind it are written in C.
In a short test with two APs and four users over 50 ticks, every user got assigned and both APs stayed healthy. We have not stress-tested it with a heavy crowd yet, so that is still on our list.
Still to come are more balancing algorithms to compare against, CSV and log files, the QoS manager, the Python dashboard and the final documentation. Overall, we are about 45% done.

Progress Overview

The main C++ core of WiBalance is up and running. It can create simulated APs and users, work out the metrics and a weighted load score for each AP, spot the congested ones and shift users over to less busy APs. A short test with two APs and four users over 50 ticks ran cleanly, with every user assigned and both APs staying healthy.
The simulation, load scoring and congestion detection are ahead of where we expected to be. Behind schedule are the extra balancing algorithms, CSV and log files, and the performance comparison, which we have only just started planning. The QoS manager, the Python dashboard, more rounds of testing and the final documentation are still to do. Real AP integration and ns-3 are optional, so we will only get to them if time allows.

Codebase Information

Repository: `https://github.com/aakriti1301chauhan-crypto/WiBalance` **Branch:** main
Language: C and C++

Files: `main.cpp`, `dsa.c`, `dsa.h`

The C++ file contains the User, AccessPoint and WiBalance classes, simulation, load balancing and menu interface. The C files provide the queue, hash table, searching and insertion sort functions used by the main program.

Important commits: (1) Add User, AccessPoint and WiBalance classes; (2) Implement queue and hash table in C; (3) Add AP load scoring and congestion detection; (4) Implement user assignment and load balancing; (5) Add menu interface and input validation.

Testing and Validation Status

Test Type	Status	Notes
Build test	Pass	Compiles with `-Wall -Wextra`, no warnings.
Simulation run (2 APs, 4 users, 50 ticks)	Pass	All users assigned; AP loads 18% and 11%, both HEALTHY.
Weighted load balancing	Pass	Users spread across both APs by distance and load.
Input validation	Pass	Bad IDs, duplicates and minimum above requested bandwidth are rejected.
Empty configuration	Pass	Start is blocked when there are no APs or users.
High-load / congestion scenario	Pending	Needs a larger number of users.
Algorithm comparison	Pending	Other algorithms not added yet.

Deliverables Progress

- **C++ Wi-Fi management system — In progress:** The core is working. It is built from C++ classes (User, AccessPoint, WiBalance) plus a C data-structure module, and it runs from a menu.
- **Simulated monitoring of APs and users — Completed:** Each AP reports load %, channel utilisation, latency and packet loss, and each user has a traffic type, demand, signal and location.
- **Weighted AP load calculation and congestion detection — Completed:** The weighted score uses load, signal, latency, client count, priority and mobility. APs are marked HEALTHY, MODERATE or CONGESTED.
- **Dynamic user distribution — Completed:** Users are re-assigned to the best AP on every tick of a 50-tick simulation.
- **DSA-based balancing algorithms — In progress:** The weighted method is working and uses a queue, a hash table, linear search and insertion sort. The other algorithms for comparison are pending.
- **Performance comparison — In progress:** Latency, packet loss and utilisation are calculated for each AP. The algorithm comparison is pending.
- **Priority-based traffic handling — In progress:** User priority is already part of the AP score. The separate QoS manager is pending.
- **Python dashboard — Pending.**
- **Final documentation — Pending.**